Criterion C: Development

This study time tracker is a desktop application that developed on Vs Code. It consists of multiple pages with a timer, a page of recorded times, and a visual representation on charts of the records presented on two different formats. It also has different modules that contribute to the build up of the presentation, the functionalities and the data management to maintain an organised/seamless application with clear algorithms, that eased the testing processes of the application. In the modules, there was a usage of HTML and CSS, to contribute to the handling of the user interactions and interfaces. Additionally there was a usage of Javascript, that contributed to the logic of the application, data processing and the state. Lastly there, another module that was used to save and collect the recorded data for the persistent data storage used.

The Timer (Algorithmic Complexity)

```
const Home = () => {
  const formatTime = (totalSeconds: number) => {
  };

  const handleStart = () => {
    setIsRunning(true);
  };

  const handlePause = () => {
    setIsRunning(false);
  };

  const handleReset = () => {
    if (seconds >= 60 && selectedSubject) {
      const sessions = JSON.parse(localStorage.getItem("study-sessions") || "[]");
      const newSession: StudySession = {
        id: Date.now().toString(),
        subject: selectedSubject,
        duration: seconds,
        date: new Date(),
      };
      localStorage.setItem("study-sessions", JSON.stringify([newSession, ...sessions]));
      toast({
        title: "Session saved!",
        description: `${formatDuration(seconds)} of ${selectedSubject} recorded.`,
      });
    }
    setIsRunning(false);
    setSeconds(0);
```

Figure 1: Asynchronous time calculation

This set of code was made for the timer. To build this timer, I used React's useEffect hook and setInterval tool was implemented. This allowed the timer's logic to be synced with React’s process. I ensured that the timer was updated using the arrow function prev => prev+1 method.

Data Structuring and Ingenuity

```

const Progress = () => {
  const [sessions, setSessions] = useState<StudySession[]>([]);
  const [selectedSubject, setSelectedSubject] = useState<string | null>(null);
  const [subjects] = useState<string[]>(DEFAULT_SUBJECTS);
  const [chartType, setChartType] = useState<"line" | "bar">("line");

  useEffect(() => {
    const saved = localStorage.getItem("study-sessions");
    if (saved) {
      setSessions(
        JSON.parse(saved).map((s: StudySession) => ({
          ...s,
          date: new Date(s.date),
        }))
      );
    }
  }, []);
```

Figure 2: Memory diagram

To enable this software to have data integrity, I decided to use a TypeScript Interface (*Documentation - Object Types*, n.d.) that enforces a structured JSON model. This model enables a clear setup for the session records minimising potential errors, across the pages so that the chart engine can also easily interpret the recorded data. In addition, the structure allows the application to handle the nested data and future updates without hindering the current filters.

Every time the app loads, it imports the data. To import the data the function `.map()` is used, that grabs the stored string and converts it back into JavaScript Date objects when the system starts. The storing of strings in the database adds convenience. Basically, I decided to split the data from how it is stored in the database to how it is in the app whilst running to be able to enhance the security, such as if the app crashes the data stored remains unaffected.

Line Chart Analytics (Data Aggregation)

```
const lineChartData = useMemo(() => {
  const filtered = selectedSubject
    ? sessions.filter((s) => s.subject === selectedSubject)
    : sessions;

  // Group by date
  const grouped: Record<string, number> = {};
  filtered.forEach((session) => {
    const dateKey = session.date.toLocaleDateString([], {
      month: "short",
      day: "numeric",
    });
    grouped[dateKey] = (grouped[dateKey] || 0) + session.duration / 3600;
  });

  // Convert to array and sort by date
  return Object.entries(grouped)
    .map(([date, hours]) => ({
      date,
      hours: parseFloat(hours.toFixed(1)),
    }))
    .slice(-10); // Last 10 entries
}, [sessions, selectedSubject]);

const barChartData = useMemo(() => {
  // Group by date, then by subject for stacked bars
  const dateMap: Record<string, Record<string, number>> = {};
```

Figure 3: Object reference

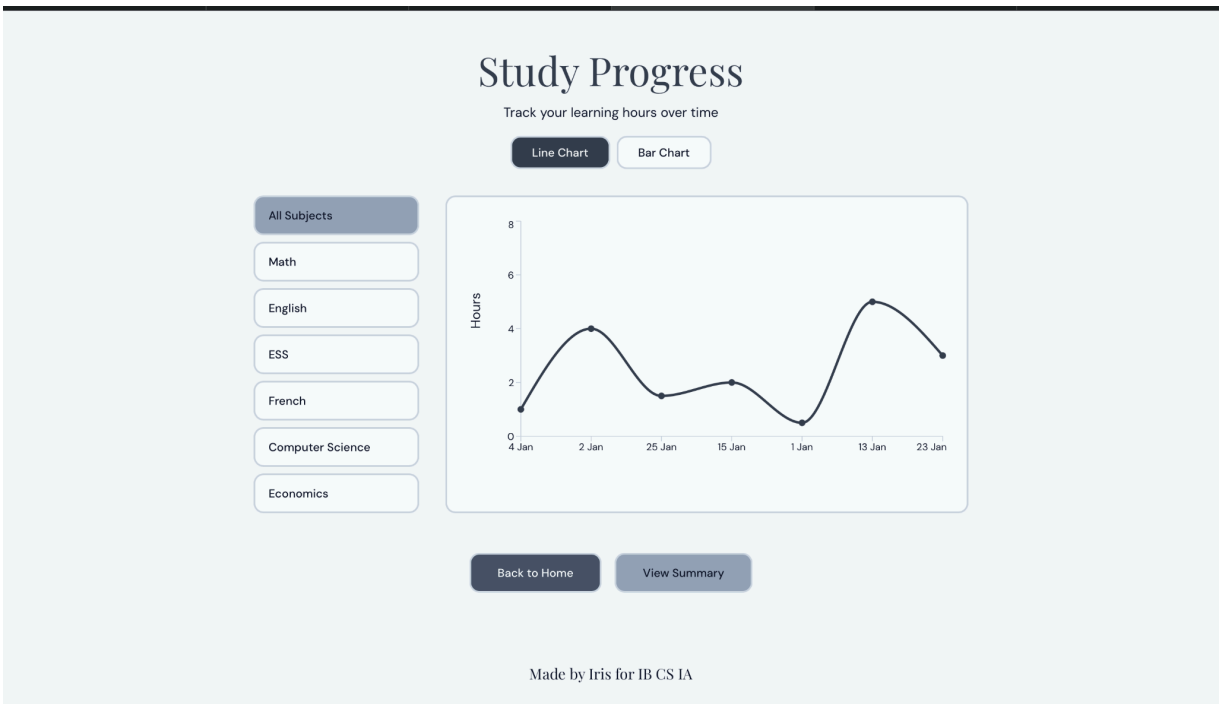

Figure 4: Graphical Representation of study hours

For the line chart graphs to function seamlessly, I wrote a time series aggregation algorithm. The function of this is to group data using a hash map before it get sent into the Recharts library (*Recharts - Npm*, 2026). Recharts library was also picked because it has a high quality API for the drawing of SVG charts. Therefore, centralising the entire focus on the O(n) math and data reduction, rather than experiencing the hassle of manually building and drawing on a canvas

Additionally, the aggregation logic was wrapped in the useMemo() hook. This was used to stop the application from continuously recalculating all input records every time the data on the user interface is having updates. On top of that, there was a verification of

the sorting of everything by unix timestamps and using a .slice() function to clip data to create a record.

Global State Managements

```
const Progress = () => {
  const [sessions, setSessions] = useState<StudySession[]>([]);
  const [selectedSubject, setSelectedSubject] = useState<string | null>(null);
  const [subjects] = useState<string[]>(DEFAULT_SUBJECTS);
  const [chartType, setChartType] = useState<"line" | "bar">("line");
```

Figure 5: Memory allocation diagram

Lastly, to maintain the data consistency throughout the entire application, I used React's centralised state architecture (*Sharing State Between Components – React*, n.d.). To maintain the app's state in a central location, every component can have easy access to it, guaranteeing that the timer and the progress charts are always interpreting the same data, and updates remain the same across, so that when the different pages are being accessed the data remain the same.

To be able to handle data safely, there was a usage of the spread operator to make unchangeable state changes. This prohibits the data from being accidently overwritten when there are multiple different operations being handled at once. Finally, with the pairing of the useEffect hook that is continuously watching the global state. Any time that there is a change, the hook syncs instantly the new state directly to the

LocalStorage. This creates a system of automatic saving process, so that the client gets a seamless experience without ever having to manually save.

References

Documentation - Object Types. (n.d.). TypeScript. Retrieved January 26, 2026, from https://www.typescriptlang.org/docs/handbook/2/objects.html

recharts - npm. (2026, January 21). NPM. Retrieved January 25, 2026, from https://www.npmjs.com/package/recharts

Sharing State Between Components – React. (n.d.). React. Retrieved January 26, 2026, from https://react.dev/learn/sharing-state-between-components